IT5702
Mobile Computing

Assignment - I

submitted by
Yuvaraj K
2022506079

Department of Information Technology
Madras Institute of Technology
Anna University, Chennai – 600 044

June – November 2025

Contents

S. No.	Title	Page No.
1	Google Android Platform	1
2	Eclipse Simulator	3
3	Android Application Architecture	6
4	Event-based Programming	10
5	Apple iPhone Platform	13
6	UI Tool Kit Interface	19
7	Cross Platform Design and Tools	23

Google Android Platform

Android is a powerful open-source operating system, and is the largest installed base among various mobile platforms across the globe. Hundreds of millions of mobile devices are powered by Android in more than 190 countries of the world. It conquered around 71% of the global market share by the end of 2021, and this trend is growing bigger every other day. The company named Open Handset Alliance developed Android for the first time. It was based on the modified version of the Linux kernel and other open-source software. Google sponsored the project at initial stages and in the year 2005, it acquired the whole company. In September 2008, the first Android-powered device was launched in the market. Android dominates the mobile OS industry because of the long list of features it provides. It's user-friendly, has huge community support, provides a greater extent of customization, and a large number of companies build Android-compatible smartphones. As a result, the market observed a sharp increase in the demand for developing Android mobile applications, and with that companies needed smart developers with the right skill set. At first, the purpose of Android was thought of as a mobile operating system. However, with the advancement of code libraries and its popularity among developers of the divergent domain, Android became an absolute set of software for all devices like tablets, wearables, set-top boxes, smart TVs, notebooks, etc.

Features of Android

- Android supports different types of connectivity for GSM, CDMA, Wi-Fi, Bluetooth, etc.
- Using Wi-Fi, we can pair with other devices while playing games or using applications.
- It contains multiple APIs to support location-tracking services such as GPS.
- We can manage all data storage-related activities by using the file manager.
- It contains a wide range of media supports like AVI, MKV, FLV, MPEG4, etc.
- It also supports different image formats like JPEG, PNG, GIF, BMP, MP3, etc.
- It supports multimedia hardware control to perform playback or recording using a camera and microphone.
- It has an integrated open-source WebKit layout-based web browser to support User Interfaces like HTML5, and CSS3.
- Android supports multi-tasking.
- It provides support for virtual reality or 2D/3D Graphics.

Android Versions

Google first publicly announced Android in November 2007 but was released on 23 September 2008, to be exact. The first device to bring Android into the market was the HTC Dream with the version Android 1.0. Since then, Google released a lot of android versions such as Apple Pie, Banana Bread, Cupcake, Donut, Éclair, Froyo, Gingerbread, Jellybeans, Kitkat, Lollipop, Marshmallow, Nougat, Oreo, etc. with extra functionalities and new features.

Programming Languages used in Developing Android Applications

1. Java
2. Kotlin

Developing the Android Application using Kotlin is preferred by Google, as Kotlin was made an official language for Android Development, which is developed and maintained by JetBrains. Previously before Java was considered the official language for Android Development. Kotlin was made official for Android Development in Google I/O 2017.

Eclipse Simulator

When it comes down to application development, a developer or mobile application development company in India mostly prefers the Android platform. In fact, there are a huge number of Android smartphone users on a global scale. Hence it is important understand what Android application development is in Eclipse.

You will be surprised to know that 49% of the application users visit Google Play Store 11 times a day. The credit behind this increasing use goes to the developers that work hard to bring innovative application ideas. However, developing applications is a challenging task for any company or individual developer. Before anything, picking the right technology comes out as their first hurdle.

There are many choices available but Android Studio and Eclipse are two of the widely used tools for Android application development. While Eclipse is no longer primarily in use for Android application development after having been… eclipsed by Android Studio, it is important to understand how applications were developed in it.

Eclipse is an Integrated Development Environment (IDE) for developing Android applications. It was first made for the Java-based application but now supports many different programming languages. This IDE features a base workspace and many plug-ins that allow it to extend its capabilities and personalize the development environment. As it's free and open source, developers used it widely for building Android applications. It can be downloaded a macOS, Windows, or Linux-based system.

Features

- Simplified Software Update System
- Easy to use Functionalities
- Plenty of bundles and plug-ins available for extending its potential
- JPA and JSF tooling support
- Ideal for creating spring-based applications
- Used in many modelling projects

Installation and Setup

Before beginning the Android application development Eclipse process, you will have to download it first on your PC from the official website:

1. Open Google Chrome on your PC
2. Open the link https://www.eclipse.org/downloads/
3. Now, click on the "Download x86_64" button

4. Start the installation and wait for it to finish

After you have installed Eclipse, it's time to prepare it for Android application development. Go through the following steps to setup this IDE for Android application development:

1) Install SDK
The eclipse will need Android SDK to work on your computer. You can download it online. Install it and move on to the next step. You can also install SDK later on in Eclipse while selecting your preferences.

2) Install New Software
- Run Eclipse and from the home interface, click on the "Help" button.
- Go to "Install New Software".
- A new window will appear.
- In the work with box, just type https://dl-ssl.google.com/android/eclipse/.
- Click "Add".
- Again, a dialog box will pop up, enter "ADT Repository" and hit the "Add" button.
- Now, tick the "developer tools" option shown under the "Name" section.
- One more dialog box will show up, click on "Next" and "Finish" to start installing the ADT plugin.

3) Setup the Preferences
Eclipse will restart when you are done with the installation of the ADT plugin. Right after the restart, a dialog box will come where you have to choose your preferences.

- Click "Open Preferences" → "Proceed".
- If the dialog box does not appear after restarting, choose "Windows" → "Preferences".
- Choose "Android" from the left panel where the software asks whether you want to send stats to Google or not.
- Click "Proceed".
- Move on to the "Browse" button and search for the SDK directory (C:\Program Files\Android\android-SDK).
- Click "Apply" > "Ok" to finish setting up your preferences.

In case you haven't configured SDK yet in your system, just click on "Install New SDK" > "Next" in Eclipse. One more dialog box will pop up, accept all the mentioned packages and click "Install". If you have Android Studio installed, you can copy the Android SDK location path from "Android" > "Tools" > "SDK Manager".

4) Create Android Virtual Machine (AVD)
When you are done with the SDK installation, you have to make an AVD so that the applications you build can be run on an Android emulator.

- Open the “Windows” menu and choose “AVD Manager”.
- Choose from the existing AVD and “Edit” or click on the “New” button to start creating an AVD.
- If you are creating a new one, enter the name of your AVD and select the Android version.
- Now, click on the “Create AVD”
- For those editing the existing AVD, they will have to edit the preference in the dialog box.

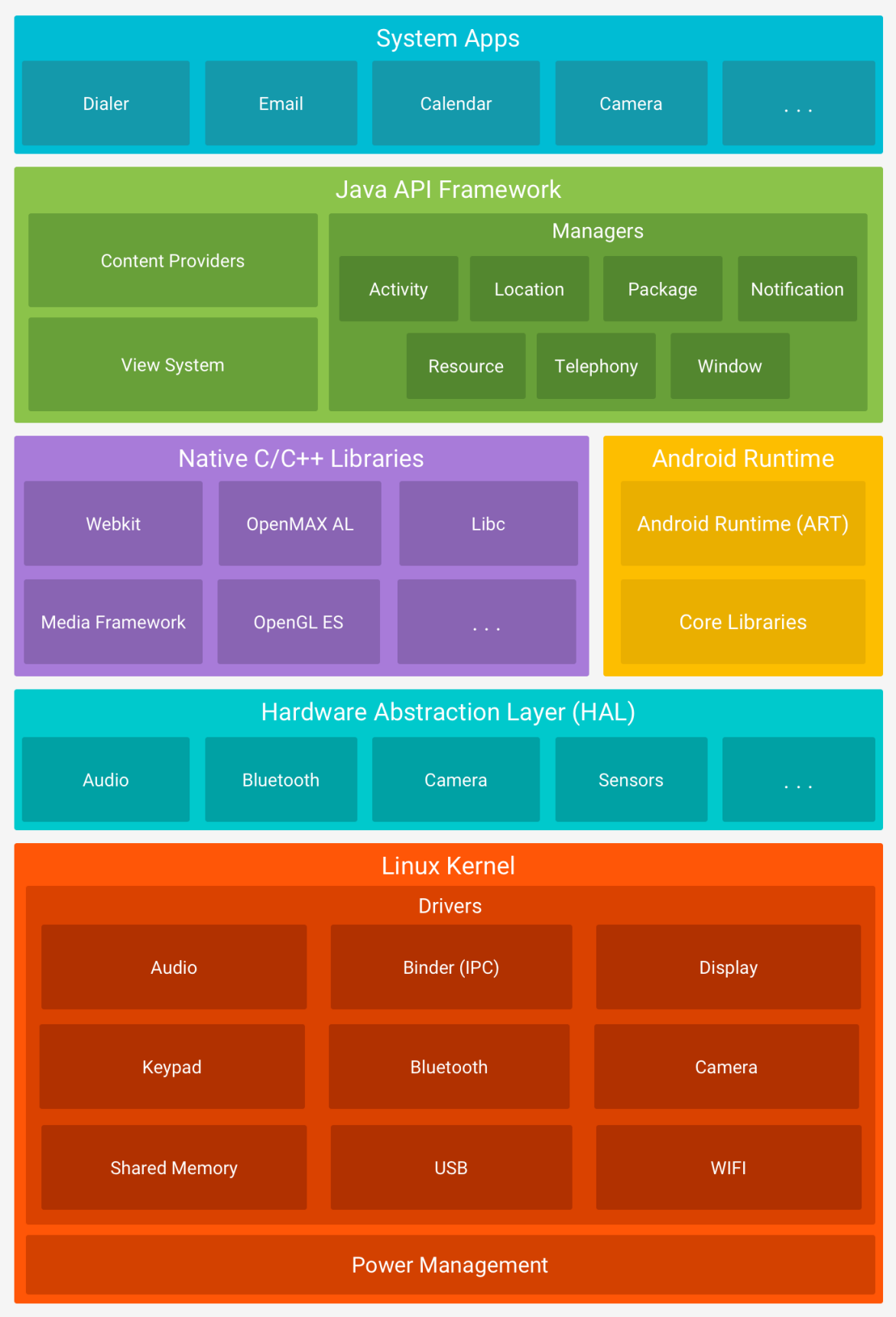
5) Build Your First Application
For building your first Android application,

- Choose “File” → “New” → “Other”.
- You will see a new dialog box.
- Choose “Android” → “Android Application Project” and click “Next”.
- You are ready now to create your first Android application in Eclipse.

While Eclipse was a prominent IDE for Android development in the past, Google officially deprecated the ADT Plugin in 2015 and transitioned to Android Studio as the primary IDE for Android development. Android Studio also includes a robust emulator for testing and debugging.

Android Application Architecture

Android is an open source, Linux-based software stack created for a wide array of devices and form factors. The below figure shows the major components of the Android platform.



Linux Kernel

The foundation of the Android platform is the Linux kernel. For example, the Android Runtime (ART) relies on the Linux kernel for underlying functionalities such as threading and low-level memory management.

Using a Linux kernel lets Android take advantage of key security features, including:

- A user-based permissions model
- Process isolation
- Extensible mechanism for secure IPC
- The ability to remove unnecessary and potentially insecure parts of the kernel

As a multiuser operating system, a fundamental security objective of the Linux kernel is to isolate user resources from one another. The Linux security philosophy is to protect user resources from one another, Linux:

- Prevents user A from reading user B's files
- Ensures that user A doesn't exhaust user B's memory
- Ensures that user A doesn't exhaust user B's CPU resources
- Ensures that user A doesn't exhaust user B's devices (for example, telephony, GPS, and Bluetooth)

Hardware Abstraction Layer (HAL)

The Hardware Abstraction Layer (HAL) provides standard interfaces that expose device hardware capabilities to the higher-level Java API framework.

The HAL consists of multiple library modules, each of which implements an interface for a specific type of hardware component, such as the camera or Bluetooth module.

When a framework API makes a call to access device hardware, the Android system loads the library module for that hardware component.

Android Runtime

For devices running Android version 5.0 (API level 21) or higher, each application runs in its own process and with its own instance of the Android Runtime (ART).

ART is written to run multiple virtual machines on low-memory devices by executing Dalvik Executable format (DEX) files, a bytecode format designed specifically for Android that's

optimized for a minimal memory footprint. Build tools, such as d8, compile Java sources into DEX bytecode, which can run on the Android platform.

Some of the major features of ART include the following:

- Ahead-of-time (AOT) and just-in-time (JIT) compilation
- Optimized garbage collection (GC)
- On Android 9 (API level 28) and higher, conversion of an application package's DEX files to more compact machine code
- Better debugging support, including a dedicated sampling profiler, detailed diagnostic exceptions and crash reporting, and the ability to set watchpoints to monitor specific fields

Prior to Android version 5.0 (API level 21), Dalvik was the Android runtime. If your application runs well on ART, then it can work on Dalvik as well, but the reverse might not be true. Android also includes a set of core runtime libraries that provide most of the functionality of the Java programming language, including some Java 8 language features, that the Java API framework uses.

Native C/C++ Libraries

Many core Android system components and services, such as ART and HAL, are built from native code that requires native libraries written in C and C++. The Android platform provides Java framework APIs to expose the functionality of some of these native libraries to applications.

For example, you can access OpenGL ES through the Android framework's Java OpenGL API to add support for drawing and manipulating 2D and 3D graphics in your application.

If you are developing an application that requires C or C++ code, you can use the Android NDK to access some of these native platform libraries directly from your native code.

Java API Framework

The entire feature-set of the Android OS is available to you through APIs written in the Java language. These APIs form the building blocks you need to create Android applications by simplifying the reuse of core, modular system components and services, which include the following:

- A rich and extensible view system you can use to build an application's UI, including lists, grids, text boxes, buttons, and even an embeddable web browser

- A resource manager, providing access to non-code resources such as localized strings, graphics, and layout files
- A notification manager that enables all applications to display custom alerts in the status bar
- An activity manager that manages the lifecycle of applications and provides a common navigation back stack
- Content providers that enable applications to access data from other applications, such as the Contacts application, or to share their own data

Developers have full access to the same framework APIs that Android system applications use.

System Applications

Android comes with a set of core applications for email, SMS messaging, calendars, internet browsing, contacts, and more. Applications included with the platform have no special status among the applications the user chooses to install. So, a third-party application can become the user's default web browser, SMS messenger, or even the default keyboard. Some exceptions apply, such as the system's Settings application.

The system applications function both as applications for users and to provide key capabilities that developers can access from their own application.

For example, if you want your application to deliver SMS messages, you don't need to build that functionality yourself. You can instead invoke whichever SMS application is already installed to deliver a message to the recipient you specify.

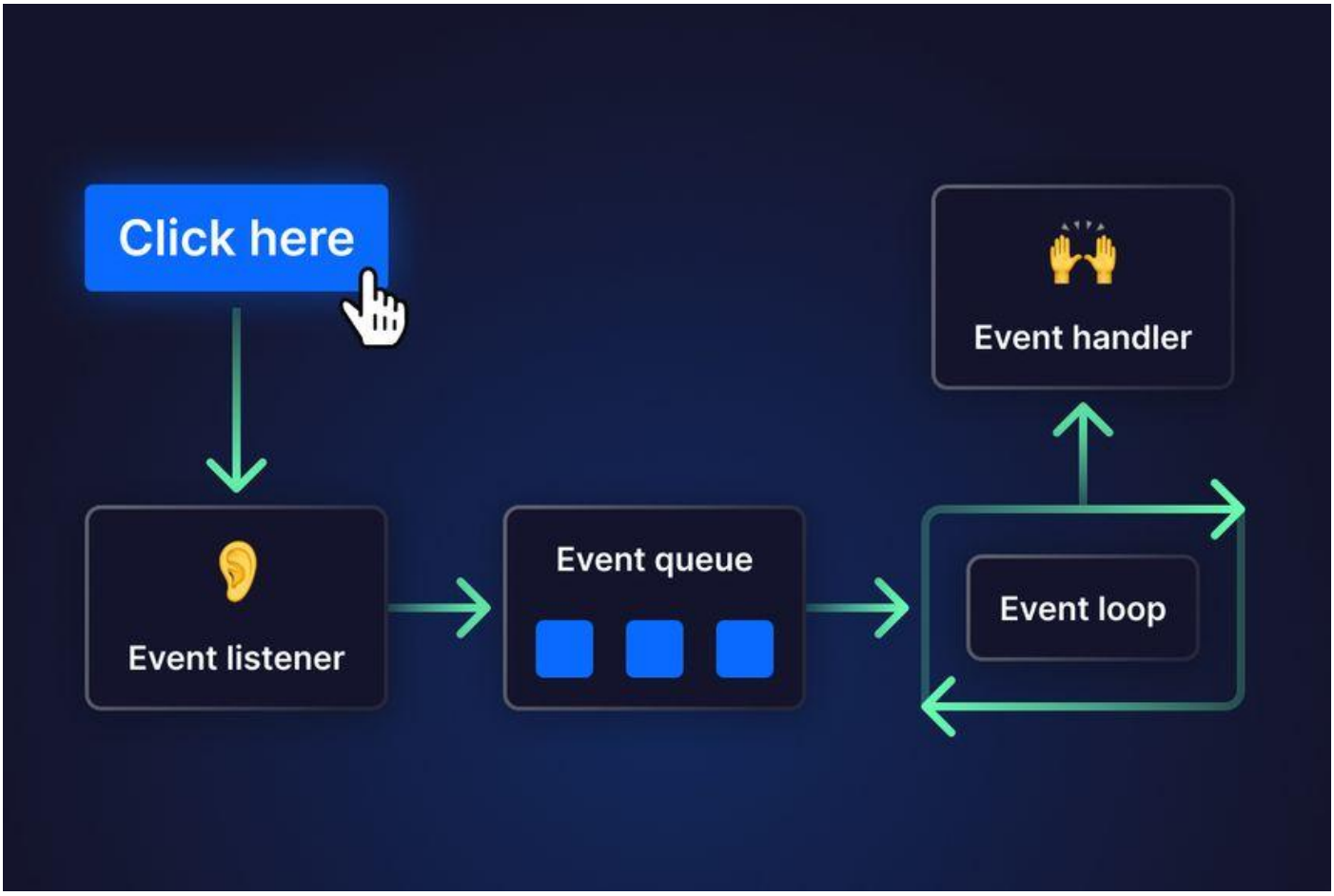
Event-based Programming

In computer programming, event-based or event-driven or event-oriented programming is a programming paradigm in which the flow of the program is determined by external events.

It is a paradigm where entities (objects, services, and so on) communicate indirectly by sending messages to one another through an intermediary. The messages are typically stored in a queue before being handled by the consumers.

Unlike in using direct calls, event-driven programming completely decouples the producer from the consumer, leading to some noteworthy benefits. For example, multiple producers and multiple consumers can all collaborate to process incoming requests. Retrying failed operations and maintaining an event history are also simplified. Event-driven programming also makes it easier to scale large systems, adding capacity simply by adding consumers.

UI events from mice, keyboards, touchpads and touchscreens, and external sensor inputs are common cases. Events may also be programmatically generated, such as from messages from other programs, notifications from other threads, or other network events.



Event-driven programming is the dominant paradigm used in graphical user interfaces applications and network servers. In an event-driven application, there is generally an event

loop that listens for events and then triggers a callback function when one of those events is detected.

Event-driven programs can be written in any programming language, although the task is easier in languages that provide high-level abstractions. Although they do not exactly fit the event-driven model, interrupt handling and exception handling have many similarities.

It is important to differentiate between event-driven and message-driven (aka queue driven) paradigms: Event-driven services (e.g. AWS SNS) are decoupled from their consumers. Whereas queue/message driven services (e.g. AWS SQS) are coupled with their consumers.

Events

Events are pieces of data that are sent by producers and eventually consumed by consumers. For example, a mouse down event typically contains the following information:

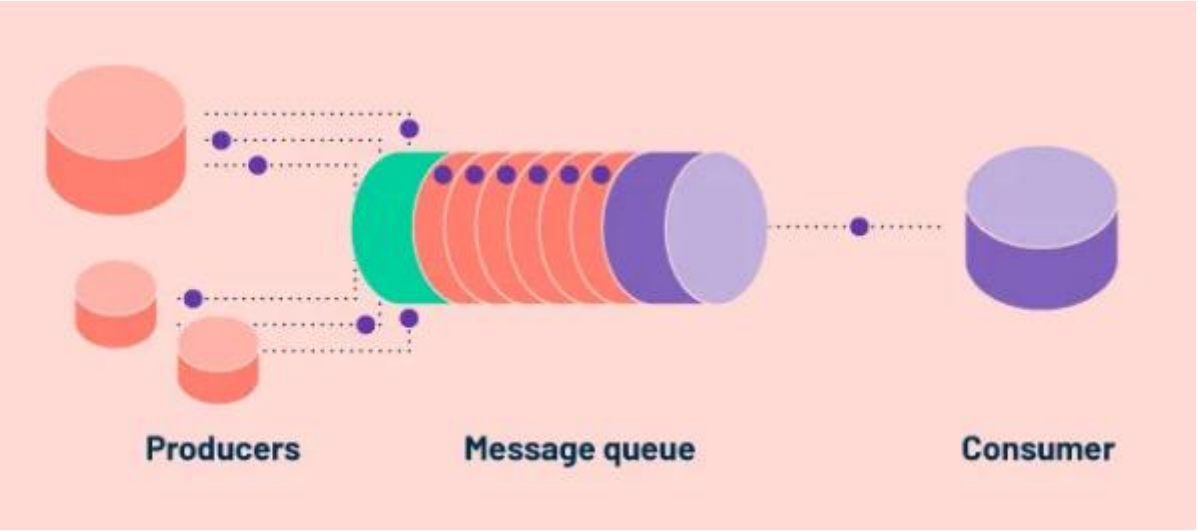
- Mouse pointer coordinates
- Which mouse buttons are pressed

Events are usually structured but can sometimes just be blobs of data (such as a chunk of JSON) that consumers should know how to parse and handle.

Producers and Consumers

Producers are entities that generate events and send them to a message queue. Consumers either subscribe to receive new events or poll periodically from the queue. Key to the event-driven paradigm is this main point: Producers and consumers are blissfully unaware of one another and interact through the message queue only.

Message Queues



Message queues are repositories of messages (events). The messages may be stored in memory and/or durable storage. Message queues are typically partitioned into topics. Producers and consumers send and receive messages for particular topics separately.

A message broker is responsible for making sure messages that are sent to the queue are delivered to all subscribed consumers. Some message brokers in popular use today include RabbitMQ, Redis, and Apache Kafka.

Consumers use pull or push mode to consume messages. In pull mode, the message queue keeps all the messages for all the consumers. When a message has been received by all consumers, that message may be removed from the queue (depending on retention configuration). Some message queues allow consumers to pull historical messages. In push mode, the message queue pushes any new message to all current subscribers.

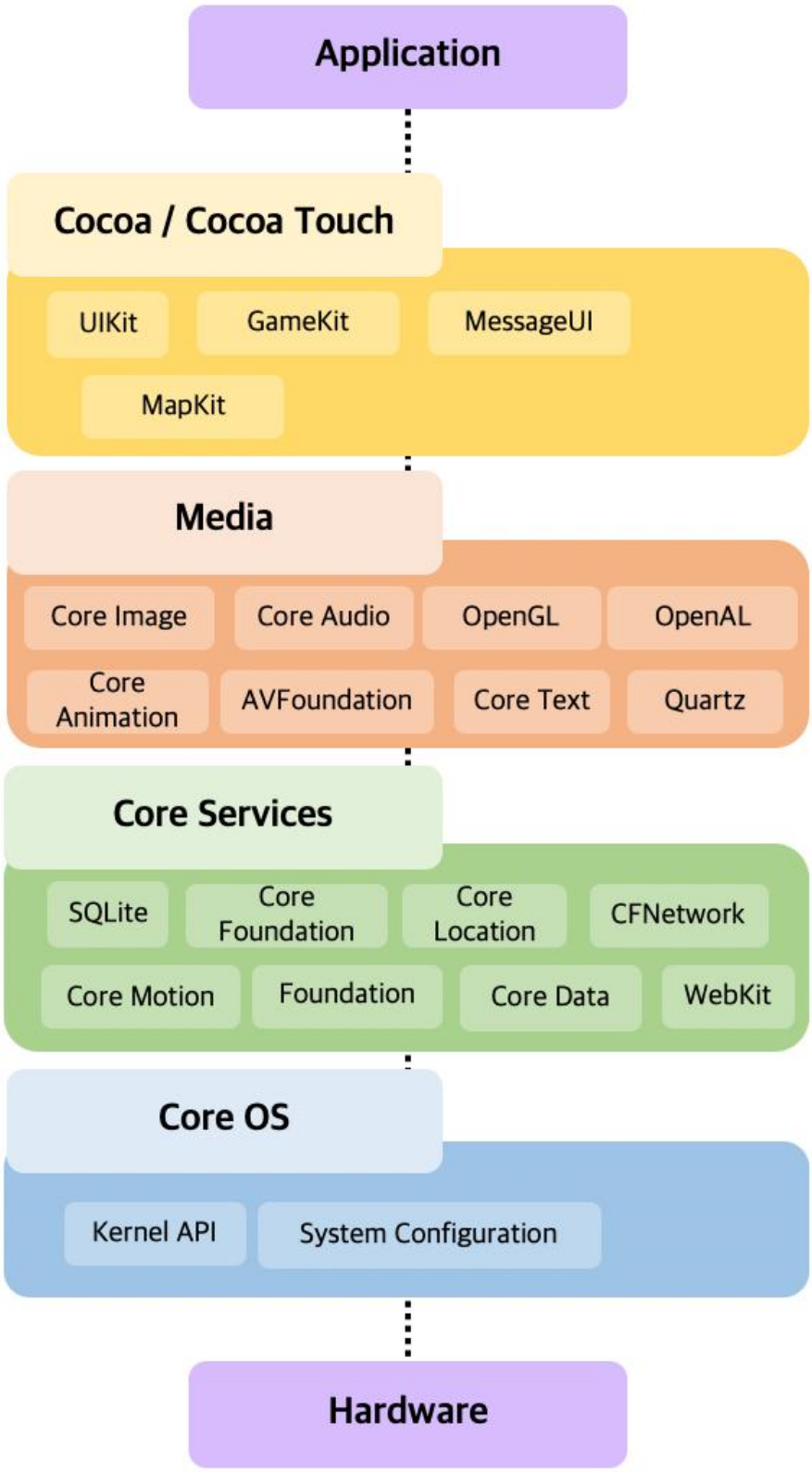
Advantages of Event-Driven Programming Paradigm

- Enables asynchronous processing, optimizing resource utilization and responsiveness, crucial for real-time applications and user interfaces.
- Encourages modular code design, simplifying maintenance and scalability by separating concerns and promoting code reusability.
- Enhances user experience by responding promptly to user inputs, delivering a smoother and more interactive interface.
- Facilitates easier integration of new features or modifications, promoting adaptability to changing requirements in dynamic environments.
- Components communicate through events, reducing dependencies and enhancing system flexibility, making it easier to maintain and modify.

Disadvantages of Event-Driven Programming Paradigm

- Event-driven systems can be challenging to debug due to their asynchronous nature, making it harder to trace errors.
- Concurrent events may introduce race conditions, leading to unpredictable behaviour and making debugging and synchronization complex.
- Event-driven systems may lead to inversion of control, making code harder to follow and understand for developers unfamiliar with the design.
- A series of interconnected events can lead to cascading effects, making it harder to predict the outcome and manage the system state.
- Continuous listening for events can consume system resources, leading to potential inefficiencies in resource utilization and impacting overall system performance.

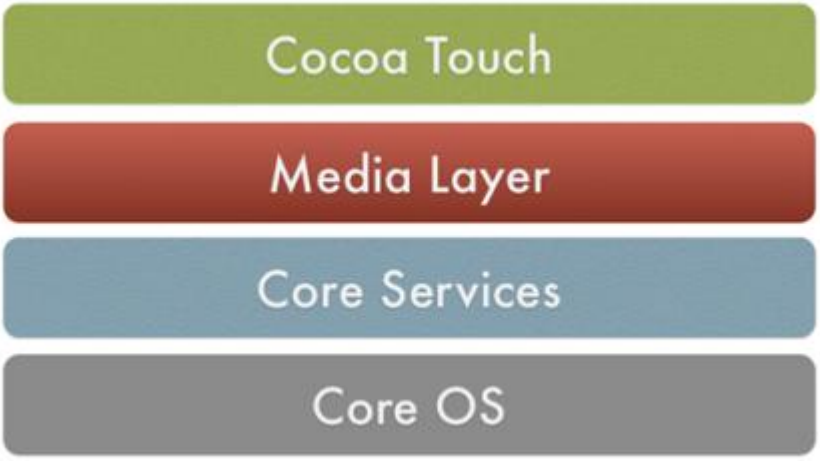
Apple iPhone Platform



iOS (formerly iPhone OS) is a mobile operating system created and developed by Apple for its iPhone line of smartphones. It was unveiled in January 2007 alongside the first-generation iPhone, and was released in June 2007. Major versions of iOS are released annually; the current stable version, iOS 18, was released to the public on September 16, 2024.

Besides powering iPhone, iOS is the basis for three other operating systems made by Apple: iPadOS, tvOS, and watchOS. iOS formerly also powered iPads until iPadOS was introduced in 2019 and the iPod Touch line of devices until its discontinuation. iOS is the world's second most widely installed mobile operating system, after Android. As of December 2023, Apple's Application Store contains more than 3.8 million iOS mobile applications.

iOS is based on macOS. Like macOS, it includes components of the Mach microkernel and FreeBSD. It is a Unix-like operating system. Although some parts of iOS are open source under the Apple Public Source License and other licenses, iOS is proprietary software.



The architecture of IOS is a layered. At the uppermost level, iOS works as an intermediary between the underlying hardware and the applications you make. Applications do not communicate with the underlying hardware directly.

Applications talk with the hardware through a collection of well-defined system interfaces. These interfaces make it simple to write applications that constantly work on devices having various hardware abilities.

Lower layers give the basic services which all application relies on and the higher level layer gives sophisticated graphics and interface-related services.

Apple provides most of its system interfaces in special packages called frameworks. A framework is a directory that holds a dynamic shared library that is .files, related resources like as header files, images, and helper applications required to support that library. Every layer has a set of Framework that the developer uses to construct the applications.

Core OS Layer

The Core OS layer holds the low-level features that most other technologies are built upon.

- Core Bluetooth Framework
- Accelerate Framework
- External Accessory Framework
- Security Services Framework
- Local Authentication Framework

Core Services Layer

Some of the important frameworks available in the core services layers are detailed:

Address book Framework	Gives programmatic access to a contacts database of user.
Cloud Kit Framework	Gives a medium for moving data between your application and iCloud.
Core data Framework	Technology for managing the data model of a Model View Controller application.
Core Foundation Framework	Interfaces that give fundamental data management and service features for iOS applications.
Core Location Framework	Gives location and heading information to applications.
Core Motion Framework	Access all motion-based data available on a device. Using this core motion framework, accelerometer-based information can be accessed.
Foundation Framework	Objective C covers too many of the features found in the Core Foundation framework.
Healthkit Framework	New framework for handling health-related information of the user.
Homekit Framework	A new framework for talking with and controlling connected devices in a user's home.
Social Framework	Simple interface for accessing the user's social media accounts.
StoreKit Framework	Gives support for the buying of content and services from inside your iOS applications, a feature is known as in-application purchase.

Media Layer

Graphics Framework

UIKit Graphics	It describes high-level support for designing images and is also used for animating the content of your views.
Core Graphics Framework	It is the native drawing engine for iOS applications and gives support for custom 2D vector and image-based rendering.
Core Animation	It is an initial technology that optimizes the animation experience of your applications.
Core Images	Gives advanced support for controlling video and motionless images in a non-destructive way.
OpenGl ES and GLKit	Manages advanced 2D and 3D rendering by hardware-accelerated interfaces.
Metal	It permits very high performance for your sophisticated graphics rendering and computation works. It offers very low overhead access to the A7 GPU.

Audio Framework

Media Player Framework	It is a high-level framework that gives simple use to a user's iTunes library and support for playing playlists.
AV Foundation	It is an Objective C interface for handling the recording and playback of audio and video.
OpenAL	An industry-standard technology for providing audio.

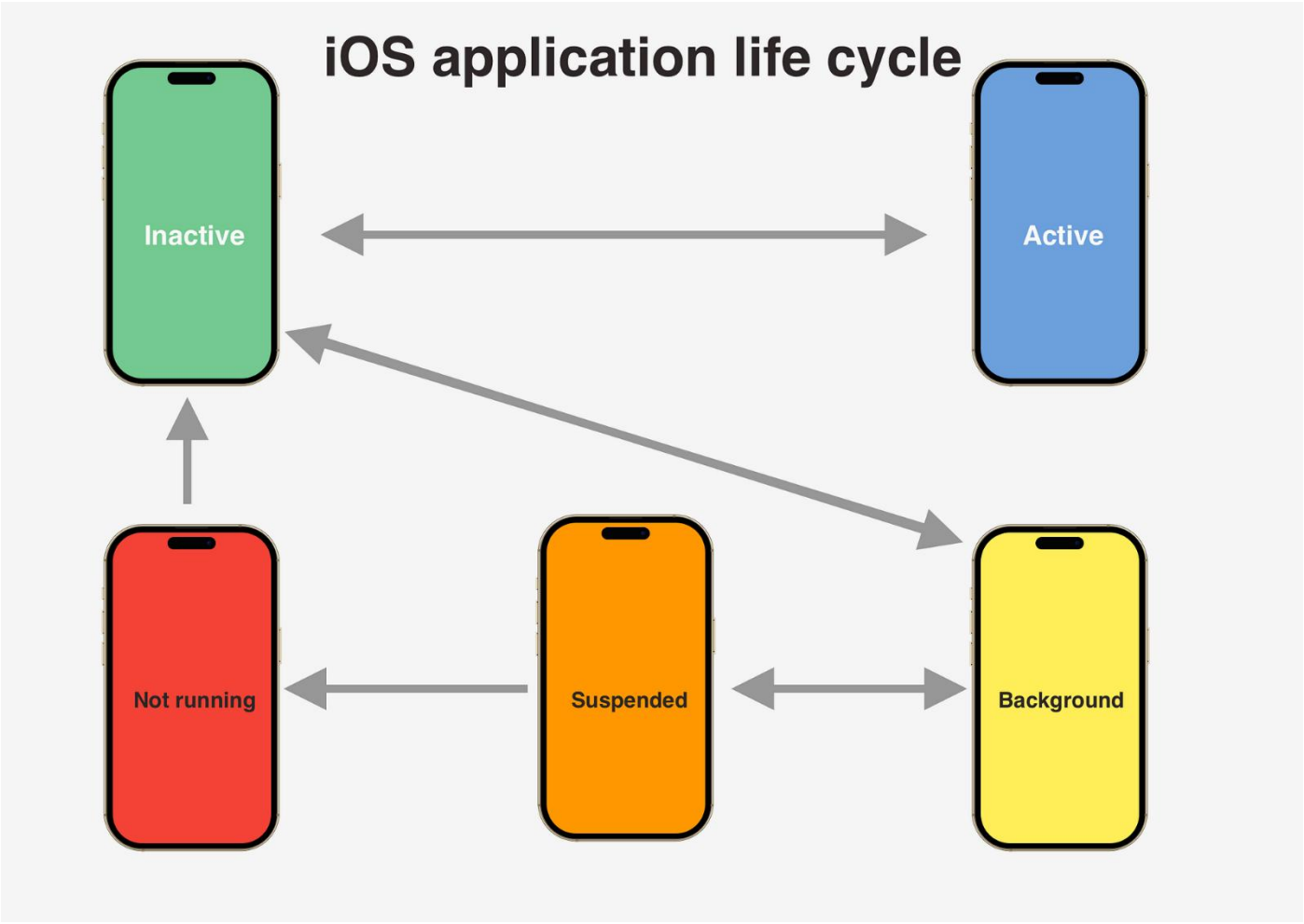
Video Framework

AV Kit	Framework gives a collection of easy-to-use interfaces for presenting video.
AV Foundation	Gives advanced video playback and recording capability.
Core Media	Framework describes the low-level interfaces and data types for operating media.

Cocoa Touch Layer

EventKit Framework	Gives view controllers for showing the standard system interfaces for seeing and altering calendar-related events.
GameKit Framework	Implements support for Game Center which allows users share their game-related information online.
iAd Framework	Allows you to deliver banner-based advertisements from your application.
MapKit Framework	Gives a scrollable map that you can include in your user interface of the application.
PushKitFramework	Provides registration support for VoIP applications.
Twitter Framework	Supports a UI for generating tweets and support for creating URLs to access the Twitter service.
UIKit Framework	Gives a vital infrastructure for applying graphical, event-driven applications in iOS. Some of the Important functions of UI Kit framework.

Life Cycle of iOS Application



The application life cycle constitutes the sequence of events that occurs between the launch and termination of application. It is very important to understand for all the iOS Developers, who wants smooth user experience.

When user turn on the phone, no applications are running except application which belong to OS. When user tap on your application icon, SpringBoard launches your application. SpringBoard is the standard application that manages the iPhone's home screen. Other tasks include starting WindowServer, launching and bootstrapping applications and setting some of the device's settings on startup

While springboard animates your application's launch screen, your application and necessary shared libraries will be loaded into the memory. Eventually your application begins execution and application delegate receives the notifications.

AppDelegate is the application delegate object. It inherits the UIResponder class and implements the UIApplicationDelegate delegate protocol. Main entry into iOS applications is UIApplicationDelegate. It is a protocol and you need to implement that into your application to get notified about user events such as application launch, application goes into background or foreground, application is terminated, a push notification was opened etc. UIResponder class make AppDelegate have the ability to respond to user events and UIApplicationDelegate enables the AppDelegate to be an application delegate object to manage and respond to the life cycle of the application.

Execution States for Applications

Not Running	The application has not been launched or terminated by the system.
Inactive	The application is entering the foreground state but not receiving events.
Active	The application enters the foreground state and can process event.
Background	In this state, if there is executable code, it will execute and if there is no executable code or the execution is complete, the application will be suspended immediately.
Suspended	The application is in the background(in memory) but is not executing code and if system does not have enough memory, it will terminate the application.

UI Tool Kit Interface

UI Toolkit is a collection of features, resources, and tools for developing user interface (UI).

Android

Jetpack Compose is Android's recommended modern toolkit for building native UI with less code, powerful tools, and intuitive Kotlin APIs. It simplifies and accelerates UI development, including adaptation to any form factor—from smartphones, foldables, and tablets to TVs and wearables.

Compose offers pre-built UI components that work with graphics, animations, and other visual elements.

Compose is an open-source Kotlin-based declarative UI framework for Android developed by Google. The first preview was announced in May 2019, and the framework was made ready for production in July 2021.

Jetpack Compose supports Android 5.0 and later. It uses the Kotlin programming language, and provides a reactive programming model similar to other UI frameworks such as Flutter, Vue.js and React Native. Compose is designed to integrate seamlessly with existing Android apps and libraries, allowing developers to gradually migrate their apps to Compose.

In Compose, a user interface is defined using functions that have been annotated with the @Composable annotation, which are known as composable functions and define the screen's state. The annotation is used by the Compose compiler to generate the UI boilerplate code. When the state is updated, composable functions are called again with new data, which causes the widgets they emit to be redrawn in a process known as recomposition. Recomposition is only performed for composable functions that need to be updated, which improves UI efficiency.

The 1.0 release introduced Compose Preview, which is built into Android Studio starting with Arctic Fox. It allows composables to be previewed using different configurations without deploying the app to a device.

Jetpack Glance is a Jetpack Compose-based framework for developing widgets for Android. Glance's first stable release took place in September 2023.

In September 2024, the 1.0 stable version of the Jetpack Compose APIs for building adaptive UIs with Material 3 was released.

iOS

UIKit provides a variety of features for building apps, including components you can use to construct the core infrastructure of your iOS, iPadOS, or tvOS apps. The framework provides the window and view architecture for implementing your UI, the event-handling infrastructure for delivering Multi-Touch and other types of input to your app, and the main run loop for managing interactions between the user, the system, and your app.

UIKit also includes support for animations, documents, drawing and printing, text management and display, search, app extensions, resource management, and getting information about the current device. You can also customize accessibility support, and localize your app's interface for different languages, countries, or cultural regions.

UIKit works seamlessly with the SwiftUI framework, so you can implement parts of your UIKit app in SwiftUI or mix interface elements between the two frameworks. For example, you can place UIKit views and view controllers inside SwiftUI views, and vice versa.

To build a macOS app, you can use SwiftUI to create an app that works across all of Apple's platforms, or use AppKit to create an app for Mac only. Alternatively, you can bring your UIKit iPad app to the Mac with Mac Catalyst.

App Structure

UIKit manages your app's interactions with the system and provides classes for you to manage your app's data and resources.

App and environment	Manage life-cycle events and your app's UI scenes, and get information about traits and the environment in which your app runs.
Documents, data, and pasteboard	Organize your app's data and share that data on the pasteboard.
Resource management	Manage the images, strings, storyboards, and nib files that you use to implement your app's interface.
App extensions	Extend your app's basic functionality to other parts of the system.
Interprocess communication	Display activity-based services to people.
Mac Catalyst	Create a version of your iPad app that users can run on a Mac device.

User Interface

Views help you display content onscreen and facilitate user interactions; view controllers help you manage views and the structure of your interface.

Views and controls	Present your content onscreen and define the interactions allowed with that content.
View controllers	Manage your interface using view controllers and facilitate navigation around your app's content.
View layout	Use stack views to lay out the views of your interface automatically. Use Auto Layout when you require precise placement of your views.
Appearance customization	Apply Liquid Glass to views, support Dark Mode in your app, customize the appearance of bars, and use appearance proxies to modify your UI.
Animation and haptics	Provide feedback to users using view-based animations and haptics.
Windows and screens	Provide a container for your view hierarchies and other content.

User Interactions

Responders and gesture recognizers help you handle touches and other events. Drag and drop, focus, peek and pop, and accessibility handle other user interactions.

Touches, presses, and gestures	Encapsulate your app's event-handling logic in gesture recognizers so that you can reuse that code throughout your app.
Menus and shortcuts	Simplify interactions with your app using menu systems, contextual menus, Home Screen quick actions, and keyboard shortcuts.
Drag and drop	Bring drag and drop to your app by using interaction APIs with your views.
Pointer interactions	Support pointer interactions in your custom controls and views.
Apple Pencil interactions	Handle user interactions like double tap and squeeze on Apple Pencil.
Focus-based navigation	Navigate the interface of your UIKit app using a remote, game controller, or keyboard.
Accessibility for UIKit	Make your UIKit apps accessible to everyone who uses iOS and tvOS.

Graphics, Drawing, and Printing

UIKit provides classes and protocols that help you configure your drawing environment and render your content.

Images and PDF	Create and manage images, including those that use bitmap and PDF formats.
Drawing	Configure your app's drawing environment using colours, renderers, draw paths, strings, and shadows.
Printing	Display the system print panels and manage the printing process.

Text

In addition to text views that simplify displaying text in your app, UIKit provides custom text management and rendering that supports the system keyboards.

Text display and fonts	Display text, manage fonts, and check spelling.
TextKit	Manage text storage and perform custom layout of text-based content in your app's views.
Keyboards and input	Configure the system keyboard, create your own keyboards to handle input, or detect key presses on a physical keyboard.
Writing Tools	Add support for Writing Tools to your app's text views.
Handwriting recognition	Configure text fields and custom views that accept text to handle input from Apple Pencil.

Cross Platform Design and Tools

In today's mobile-driven world, businesses must cater to both iOS and Android users to reach a broad audience. Cross-platform mobile application development has emerged as a powerful solution to this challenge, enabling developers to create apps that work seamlessly across multiple platforms using a single codebase. This approach not only saves time and resources but also ensures consistent user experiences across devices.

With the rise of efficient frameworks like React Native, Flutter, and Xamarin, cross-platform development is becoming increasingly popular for businesses looking to expand their reach without compromising on quality.

With Android and iOS being the two primary operating systems for mobile devices, developers are often required to create separate apps for each platform. This can be time-consuming and costly. Cross-platform mobile development addresses this challenge by allowing developers to create a single app that works seamlessly on multiple platforms. This approach saves time and resources, making it easier for businesses to reach a broader audience without compromising on quality or user experience.

As mobile usage continues to grow, the demand for cross-platform solutions will only increase. This emphasizes cross-platform mobile development as a crucial strategy for modern app development.

When it comes to mobile app development, businesses often face a crucial decision: Should they opt for native development or cross-platform development?

Aspect	Native Development	Cross-Platform Development
Codebase	Separate codebases for each platform (iOS, Android)	Single codebase shared across multiple platforms
Development Languages	Swift, Objective-C (iOS), Java, Kotlin (Android)	JavaScript (React Native), Dart (Flutter), C# (Xamarin), HTML/CSS/JS (Ionic)
Performance	Optimal performance; fully utilizes platform-specific features	Generally good performance, though slightly lower than native in some cases
Access to Native Features	Full access to all native APIs and device-specific features	Limited access; may require platform-specific code or plugins
Development Time	Longer development time due to separate coding for each platform	Shorter development time; code can be reused across platforms
Cost	Higher development cost due to separate teams or extended time	Lower development cost; single team/codebase for multiple platforms
User Experience (UX)	Superior, as the app is tailored to each platform's design guidelines	Consistent, but may not fully match platform-specific UX nuances
Maintenance	More complex; updates and fixes must be applied to each codebase	Easier maintenance; single codebase means updates apply to all platforms
Suitability	Best for high-performance apps requiring extensive native features	Ideal for apps that need to run on multiple platforms with shared features
Examples	Instagram (iOS, Android separate)	Facebook, Airbnb, Uber (initially used React Native for some features)

Popular Cross-Platform App Development Frameworks

1. Flutter

Released by Google in 2017, Flutter is a popular framework for building mobile, web, and desktop apps from a single codebase.

Programming language: Dart.
Mobile app examples: eBay Motors, Alibaba, Google Pay, ByteDance apps.

Key Features:

- Flutter's hot reload feature allows you to see how your application changes as soon as you modify your code, without you having to recompile it.
- Flutter supports Google's Material Design, a design system that helps developers build digital experiences. You can use multiple visual and behavioural widgets when building your app.
- Flutter doesn't rely on web browser technology. Instead, it has its own rendering engine for drawing widgets.
- Flutter has a relatively active community of users around the world and is widely used by many developers. According to the Stack Overflow Trends, the usage of Flutter has been trending up over time, based on the increasing use of the corresponding tag.

2. React Native

An open-source UI software framework, React Native was developed in 2015 (a bit earlier than Flutter) by Meta Platforms, formerly Facebook. It's based on Facebook's JavaScript library React and allows developers to build natively rendered cross-platform mobile apps.

Programming language: JavaScript.
Mobile app examples: React Native is used in Microsoft’s Office, Skype, and Xbox Game Pass; Meta’s Facebook, desktop Messenger, and Oculus.

Key Features:

- Developers can see their changes in their React components immediately, thanks to the Fast Refresh feature.
- One of React Native's advantages is a focus on the UI. React primitives render to native platform UI components, allowing you to build a customized and responsive user interface.
- In versions 0.62 and later, integration between React Native and the mobile app debugger Flipper is enabled by default. Flipper is used to debug Android, iOS, and React native apps, and it provides tools like a log viewer, an interactive layout inspector, and a network inspector.

As one of the most popular cross-platform app development frameworks, React Native has a large and strong community of developers who share their technical knowledge. Thanks to this community, you can get the support you need when building mobile apps with the framework.

3. Kotlin Multiplatform

Kotlin Multiplatform (KMP) is an open-source technology built by JetBrains that allows sharing code across platforms while retaining the benefits of native programming. It enables developers to reuse as much code as they want, write native code if needed, and seamlessly integrate shared Kotlin code into any project.

Programming language: Kotlin.
Mobile app examples: McDonald's, Netflix, Forbes, 9GAG, Cash App, Philips. Read more about Kotlin Multiplatform case studies.

Key Features:

- Developers can reuse code across Android, iOS, web, desktop, and server-side while keeping native code if needed.
- Kotlin Multiplatform can be seamlessly integrated into any project. Developers can utilize platform-specific APIs while making the most of both native and cross-platform development.
- Thanks to Compose Multiplatform, a modern declarative cross-platform UI framework by JetBrains, developers have full code-sharing flexibility and the ability to share both the logic and the UI.
- There's no need to introduce a new language to your codebase when you already use Kotlin for Android. You can reuse your Kotlin code and expertise, which makes migrating to Kotlin Multiplatform less risky compared to other technologies.

Even though this cross-platform mobile development framework is one of the youngest on our list, it has a mature community. In November 2023, JetBrains promoted it to Stable. It's growing fast and is already making a distinct impression on today's market. Thanks to its regularly updated documentation and community support, you can always find answers to your questions. What's more, many global companies and startups already use Kotlin Multiplatform to develop multiplatform apps with a native-like user experience.

4. Ionic

Ionic is an open-source mobile UI toolkit that was released in 2013. It helps developers build cross-platform mobile applications from a single codebase using web technologies, like HTML, CSS, and JavaScript, with integrations for the Angular, React, and Vue frameworks.

Programming language: JavaScript.
Mobile app examples: T-Mobile, BBC (Children's & Education apps), EA Games.

Key Features:

- Ionic is based on a SaaS UI framework designed specifically for mobile OS and provides multiple UI components for building applications.
- The Ionic framework uses the Cordova and Capacitor plugins to provide access to device's built-in features, such as the camera, flashlight, GPS, and audio recorder.
- Ionic has its own command-line interface, Ionic CLI, which serves as the go-to tool for building Ionic applications.

There's constant activity on the Ionic Framework Forum, where community members exchange knowledge and help each other overcome their development challenges.

5. .NET MAUI

.NET Multi-platform App UI (.NET MAUI) is a cross-platform framework that was released in May 2022 and is owned by Microsoft. It allows developers to create native mobile and desktop apps with C# and XAML. .NET MAUI is an evolution of Xamarin.Forms, one of the functionalities of Xamarin, which provides native controls for the platforms supported by Xamarin.

Programming languages: C#, XAML.
Mobile app examples: NBC Sports Next, Escola Agil, Irth Solutions.

Key Features:

- .NET MAUI provides cross-platform APIs for accessing native device features, like the GPS, accelerometer, and battery and network states.
- There's a single project system which is enabled using multi-targeting to target Android, iOS, macOS, and Windows.
- With the support for .NET hot reload, developers can modify their managed source code while the app is running.

Even though .NET MAUI is still a relatively new framework, it has already gained traction among developers and has an active community on Stack Overflow and Microsoft Q&A.

6. NativeScript

This open-source mobile application development framework was initially released in 2014. NativeScript allows you to build Android and iOS mobile apps using JavaScript or languages that transpile to JavaScript, like TypeScript, and frameworks like Angular and Vue.js.

Programming language: JavaScript, TypeScript.
Mobile app examples: Daily Nanny, Strudel, Breethe.

Key Features:

- NativeScript allows developers to easily access native Android and iOS APIs.
- The framework renders platform-native UIs. Apps built with NativeScript run directly on a native device without relying on WebViews, a system component for the Android OS that allows Android applications to show content from the web inside an app.
- NativeScript offers various plugins and pre-built app templates, eliminating the need for third-party solutions.

NativeScript is based on well-known web technologies like JavaScript and Angular, which is why many developers choose this framework. Nevertheless, it's usually used by small companies and startups.

There are other cross-platform frameworks besides those mentioned above, and new tools will continue to appear on the market. Given the wide array of options, how can you find the right one for your project? The first step is to understand your project's requirements and goals, and to get a clear idea of what you want your future app to look like. Next, you'll want to take the following important factors into account so you can decide on the best fit for your project.

1. The expertise of your team

Different cross-platform mobile development frameworks are based on different programming languages. Before adopting a framework, check what skills it requires and make sure your team of mobile engineers has enough knowledge and experience to work with it.

For example, if your team is equipped with highly skilled JavaScript developers and you don't have enough resources to introduce new technologies, it may be worth choosing frameworks that use this language, such as React Native.

2. Vendor reliability and support

It's important to be sure that the maintainer of the framework will continue to support it in the long run. Learn more about the companies that develop and support the frameworks you're considering, and take a look at the mobile apps that have been built using them.

3. UI customization

Depending on how crucial the user interface is for your future app, you may need to know how easily you can customize the UI using a particular framework. For example, Kotlin Multiplatform offers full code-sharing flexibility with Compose Multiplatform, a modern declarative cross-platform UI framework by JetBrains. It enables developers to share UIs across Android, iOS, web, and desktop (via the JVM) and is based on Kotlin and Jetpack Compose.

4. Framework maturity
Find out how frequently the public API and tooling for a prospective framework change. For example, some changes to native operating system components break internal cross-platform behavior. It's better to be aware of possible challenges you may face when working with the mobile app development framework. You can also browse GitHub and check how many bugs the framework has and how these bugs are being handled.

5. Framework capabilities
Each framework has its own capabilities and limitations. Knowing what features and tools a framework provides is crucial to identifying the best solution for you. Does it have code analyzers and unit testing frameworks? How quickly and easily will you be able to build, debug, and test your app?

6. Security
Security and privacy are especially important when building a critical mobile app for business, such as banking and e-commerce apps that include a payment system. According to OWASP Mobile Top 10, some of the most critical security risks for mobile applications include insecure data storage and authentication/authorization.

You need to ensure that the multiplatform mobile development framework of your choice provides the required level of security. One way to do this is to browse the security tickets on the framework's issue tracker if it has one that's publicly available.

7. Educational materials
The volume and quality of available learning resources about a framework can also help you understand how smooth your experience will be when working with it. Comprehensive official documentation, online and offline conferences, and educational courses are a good sign that you will be able to find enough essential information about a product when you need it.